SESSION 13: Automatic Programming and Compilers II

Moderator: B. Gilchrist

IBM Corporation, New York, N. Y.

WPM 13.1: The COLASL Automatic Coding System

K. G. Balke and G. Carter

Los Alamos, Scientific Lab.-UCLA

Los Alamos, New Mexico

RECENT YEARS have been the development of two somewhat similar automatic coding systems at the Los Alamos Scientific Laboratory of the University of California. The *MADCAP* system, generating codes to be run on the *MANIAC* computer, has been produced primarily by *Mark Wells* and *Donald Bradford* [1]. The *COLASL* system, which compiles codes to be run on the IBM-7030 (*STRETCH*) computer, is the result of work done by the authors under the direction of *Edward A. Voorhees*. Both systems have attempted to utilize source languages which are as natural as possible within the limitations of time and hardware.

Basically, the ultimate *natural* language is that in which there is a probability close to one that a scientist who knows nothing about the language will automatically state his problem in it. This is certainly a fascinating and desirable goal; however, it will be some time before such a language is implemented.

Then in what other way might a language for computers be justifiably termed *natural*? Another definition could well be that generally, once a problem has been stated in the language, any scientist who is able to read the literature pertaining to the problem can read the statement of the problem, even though he is totally unacquainted with the automatic coding language. This implies that the automatic coding language be a subset of the reflex language of the scientist using it, augmented by those elements of mathematical notation whose semantic content can be coded for a computer. It is this path which has been taken in setting up specifications for *COLASL*.

First, it was necessary to have a device which could produce two representations of the language: one, such as a deck of cards or a paper tape, which could easily be read by a computer; the other which could be easily read by humans. A combination of a card punch and an electric typewriter would satisfy both requirements. When, in addition, this device was provided with an expanded character set, with color shift and half-line platten rotation under keyboard control, the first step in the design and implementation of *COLASL* was complete. The machine, produced by IBM and called the IBM-9210 *Scientific Descriptive Printer* [2], was accepted by the *LASL* in May, 1961. This device consists of an IBM-026 printing card punch, modified to punch more than three holes per card column, connected to a special IBM triple-case typewriter having a platen which may be rotated by half a line in either direction under keyboard control.

The features of this device were exploited in four ways. First, the basic character set, shown in Figure 1, was augmented by the addition of several *constructable* characters, produced by printing one character over another. These characters are illustrated in Figure 2. To this already large set was added the ability to use any letter with the large box or circle circumscribed and/or with certain modifying characters typed one-half line above the base letter. This large literal alphabet made possible a simplification of the form of variable names, yet provided a wide choice of symbols for programming purposes. The simplification, in turn, facilitated the inclusion of implicit multiplication in the algebraic subset of the *COLASL* language. A form of descriptive subscripting was also adopted, to increase further the mnemonic capability of the system variables. Examples of some typical variable names are given in Figure 3.

Second, the platen rotation was utilized to allow a natural form of subscript and superscript notation. Going one step further, the ability to write displayed divisions and construct extended radicals was provided. Both of these features, illustrated in Figure 4, were able to add greatly to the naturalness of the algebraic notation.

Third, the presence of colons, semicolons, question marks and other sentence dividers, plus the availability of both upper and lower case Roman letters, led directly to a form of notation closely analogous to the normal and natural method of sentence construction. The sentence was taken as the basic unit of information and was designated as the only structure in the *COLASL* language which could be named. Each sentence could, in turn, be subdivided into smaller sentence units, closely resembling phrases and clauses, which, through key words and contexts, could be used to transmit information to the compiler. Several examples of sentence structure are shown in Figure 5.

In any language based upon a key word concept, and particularly in a natural notation which seeks to impose as few restrictions as possible, the problem of misinterpretation of input due to an inadvertent misuse of a key word eventually arises. The color shift of the *COLASL* input device was admirably suited to methods for alleviating, if not entirely eliminating, this problem. In *COLASL*, any material written in the red mode is not interpreted and thus has no meaning for the compiler. Thus a programmer may expound at great length, if he so desires, or may use formulas and key words in any context for explanatory or descriptive purposes without fear of being misinterpreted by the compiler. A further advantage of the color shift is that only the material directly pertaining to the execution of the problem need be written in black; hence, in debugging, the programmer is not forced to plow through paragraphs of explanatory material but can see at a glance that input which is directly pertinent to the problem's execution.

All the notational difficulties, inherent in the design and production of a natural compiler language, have not been solved. In practice, some compromise with this ideal is necessary to permit implementation of the language before the computer for which it is intended is replaced by a newer model. However, it is felt that the *COLASL* system, currently under field test at the Los Alamos Scientific Laboratory, is a large step towards such a notation.

[1] Wells, Mark B., MADCAP: A Scientific Compiler for a Displayed Formula Textbook Language," *Comm. ACM*, p. 31; Jan., 1961.

[2] International Business Machines, Inc., "Scientific Descriptive Printer Reference Manual," *IBM* form R60-0034.

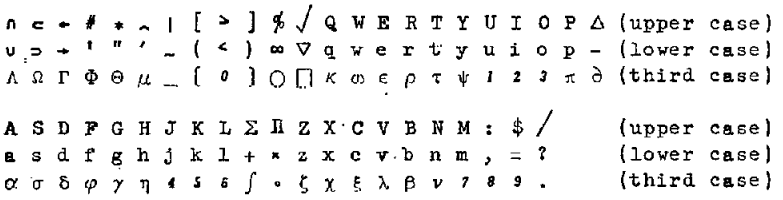


Figure 1—The 132 characters printable by the IBM 9210 *Scientific Descriptive Printer.*



Figure 2 — Common combination or *constructable* characters.

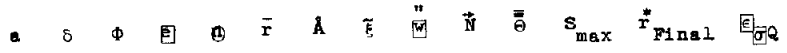


Figure 3—Examples of variable names in the *COLASL* language.

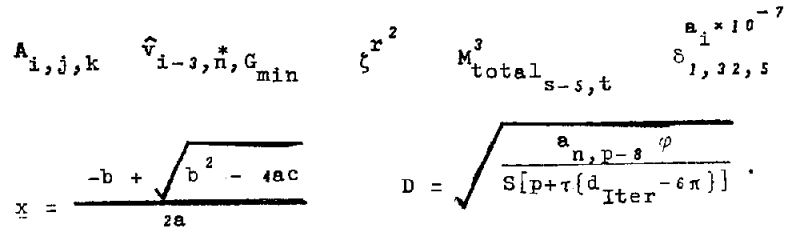


Figure 4—Examples of scripting, displayed division, and radication acceptable as input to *COLASL.*

```
Let a_i = 2b^2_{i,j} - c_i,  a_j = 6b^3_{j,i} + c_j, for (i=-1,2,...,I).
If [ c̃ ≠ d_max-N×r ] then set m̂ = {νr^2 - 7.8932}^-n, proceed from #7.
The ranges on input variables are: -1 to I for α ; 6 to 45,
A to N, and 27 to -63 for β, ω_in, and Ē .
```

Figure 5—Some examples of typical *COLASL* input sentences.